Study guide: Paper 003

Version 0.1.0, 2026-10-02. **AuthorityEffect:** None.

Read the fictional Apartment 50 preface. The quiz tests calculations and evidence limits. Its answers live in the separate instructor key, so this guide can be used before the examination.

What you should be able to do

A detector's output is a finding under its executable predicate. Admission allows a frozen policy to act on that finding within a declared route. Verification asks whether a witness satisfies the route's contract. Those are three different operations. A valid proposal can contain a flagged phrase; an internally consistent receipt can describe a wrong verdict; a missing checker can refuse without judging the proposed mathematics.

Level	Read in the manuscript	Demonstrate
0	Sections 1 and 3	Separate findings, actions, and witness verdicts; build a valid near miss
1	Section 3, Section 4.1, Appendix A.1-A.2	Compute both gate variants and diagnose a derived second signal
2	Sections 4-5	Trace rollback, nonrefundable fuel, and terminal behavior
3	Sections 4 and 8	Bind evidence to source bytes and distinguish hash consistency from authentication
4	Appendix A.3	Derive a sum threshold and state what a sensor cap proves
5	Appendix A.4-A.6	Prove the predicate implications and expose an incomplete self panel
6	Section 3.1	Separate sign gauge, local particle-hole symmetry, and determinant positivity
7	Sections 6-9	Specify the comparison, denominators, uncertainty, and stopping conditions

All quiz carriers are declared locally. The exercises need neither another paper's grammar nor a repository-only graph worksheet.

1. Build the contract before interpreting a finding

Write down the subject schema, route and checker, allowed transitions, detector predicate, defect code, and proposed action. A positive control must reproduce the named defect with the checker. A valid near miss must satisfy the same applicable contract while challenging the detector's boundary.

For example, a contract requiring a field named `witness` can reject a record without it. A nearby record containing the field with an unusual permitted value is a useful negative control if the checker accepts it. A record legal only under a different schema demonstrates a scope boundary; it cannot substitute for a negative control inside the admitted schema.

Freeze predicate, action, schema, route, checker, and fixture identities together. Hashing an unchecked assertion preserves the assertion's bytes; admission still requires its semantic check. One valid near miss is a regression control, not a distributional safety estimate.

2. Make the two tables explicit

Use `M` for at least one pattern match. In the baseline/Appendix A gate, `D` means positive defect count. In the package gate, a supplied Signal 2 record counts as present even with zero defects.

M	D	Severity when D is present	Baseline/Appendix A decision	Refuses?
0	0	-	TOLERATED	No
1	0	-	DIAGNOSTIC_ONLY	No
0	1	LOW/MEDIUM	DIAGNOSTIC_ONLY	No
0	1	HIGH/CRITICAL	DEFENSIVE_LOCKDOWN	Yes
1	1	LOW/MEDIUM	DEFENSIVE_LOCKDOWN	Yes
1	1	HIGH/CRITICAL	QUARANTINED	Yes

The package labels both dual-signal cases QUARANTINED. Neither table is a strict conjunction: a severe structural record alone refuses. A historical corrected gate in Appendix A.5 has a different policy; do not silently replace the audited table with it.

Suppose a system converts the pattern count directly into a HIGH defect count. The gate now decides only from that count. It cannot supply independent corroboration, even though the two values travel in separate fields. Independence here concerns how the checks are derived from the same proposal. It does not

mean their observed outcomes must be statistically uncorrelated. Constructing all input combinations in a test demonstrates the table, not the provenance of production signals.

3. Keep two forms of state in the transaction trace

Semantic state consists of coverage, proposed witness, and phase. Attempt accounting includes consumed fuel and receipt history. A refused transaction restores the former while recording the latter.

Try a package host with fuel 3. First cover an element in a valid batch: fuel becomes 2. Next try an unknown move: its semantic changes are rolled back and fuel becomes 1. A final VERIFY reaches the missing port, spends the last unit, and ends the package host in TERMINAL. No route witness has been accepted. The canonical baseline records exhaustion on the next attempt instead.

This fuel bound is a bound on charged batches. Serialization can inspect a large batch before refusal; further terminal calls can still produce receipts. Byte, move, time, and receipt-retention limits require their own declarations.

4. Treat source pins as part of the answer

The deposit has three distinct kinds of source: its standalone illustrative code; the canonical host at the paper's baseline; and the older immune-system audit/correction history in Appendix A. List each separately in a replay. A historical claim about a pinned commit remains a historical claim even when the default branch has moved.

The package receipt checker recomputes chains and refuses certificate labels, but does not replay fuel or arm scripts. Its canonical counterpart checks more protocol fields. Both need an independently held anchor to detect a complete rewrite; neither unkeyed hash chain authenticates a producer. Hash coverage, semantic replay, and producer authentication are separate properties.

Record the exact command, tool version, input digests, exit status, and skipped scope. "Not run" and "unavailable" are results of the audit, not successful verification. The archive verifier checks file custody and structure.

5. Derive the Hill threshold using exact arithmetic

Aggregate only positive entries:

$$\theta = \sum_{s_i>0} s_i, \qquad H(\theta) = \frac{\theta^n}{K^n + \theta^n}.$$

For `0<t<1`, activation is equivalent to `theta^n (1-t) >= t K^n`. This form avoids computing a root. At `t=1/2` the boundary is simply `theta>=K`.

As a practice example, `n=3,K=3,t=1/2` with signals `[1,1/2]` gives `theta=3/2` and `H=1/9`, so it does not activate. One signal of 3 does activate. A cap of 1 on every entry would instead require at least three positive entries. Counting distinct independent witnesses would also require identities, deduplication, and provenance.

The mathematical proof allows positive real `n`. The exact package helper accepts positive integer `n`, and integer or Fraction signals and scales. Python's Boolean subtype of integer does not make `True` an admissible coefficient. Negative signals are ignored here; they do not veto a positive sum. Check threshold endpoints separately.

6. Prove the screen's property before measuring it

Let `P` be the screened predicate, `E` the executed predicate, and `S` the valid self panel. Admission means no member of `S` satisfies `P`. Panel soundness means an admitted detector never fires under `E` on `S`.

If `E implies P`, admission makes firing on the panel impossible. If that implication fails at an input `x`, the singleton `S={x}` is an immediate counterexample. Avoiding rejection of a harmless rule requires the reverse implication. Both properties together require `E=P`.

Even equality does not supply missing valid inputs to the panel. With `E=P` matching only `novel` and a panel containing only `known`, admission says nothing about whether an independent route checker would accept `novel`. Executing the right predicate and covering valid cases are separate obligations. Duplicating panel rows improves neither coverage nor independence.

7. Keep the three physics questions separate

A diagonal sign gauge asks for vertex signs making a particular real symmetric matrix's nonzero off-diagonal entries nonpositive. Its signed cycle product, not the graph's odd-cycle topology alone, is the obstruction.

The standard local particle-hole transformation on a real-hopping Hubbard graph requires opposite local signs on every bond. That requirement is about bipartiteness. It differs from the gauge condition because the transformation rule fixes an opposite-sign constraint regardless of the bond's sign.

Attractive Hubbard determinant QMC instead uses equal real spin-sector determinants under the stated spin-independent and spin-balanced assumptions: their product is a square. These conditions can hold on a non-bipartite lattice. This proves neither the particle-hole symmetry above nor a generic efficient solution to the NP-hard sign-problem class. Majorana and meron methods require their own model-specific conditions; changing notation or matching a phrase does not check them. See the manuscript's primary references.

8. Choose denominators before looking at outcomes

Effectiveness is verified-witness runs divided by all allocated runs under the common budget. Keep refusals, unknowns, timeouts, and exhaustion visible. False suppression is blocked independently valid eligible proposals divided by all independently valid eligible proposals. Invalid proposals cannot make that fraction smaller.

For zero failures in independent Bernoulli opportunities, solve `(1-p)^n=alpha` to get upper bound `1-alpha^(1/n)`. At a fixed one-sided 95% confidence level, `alpha=0.05`. Repeated proposals from a shared fixture can be dependent, so the number of stored records is insufficient to select `n`. Route-specific intervals also do not automatically form a joint 95% interval; the analysis must declare its multiplicity and dependence handling.

The principal experiment compares D with matched generic reflection R. Freeze the detector bank before held-out collection. Keep discovery, admission, pilot, and confirmatory instances disjoint beneath surface relabeling. An assisted arm establishes performance with that assistance. The missing route seam still prevents this deposit's demonstrations from supplying a treatment effect.

Local replay

From this deposit's root, install the test tools and run:

```
py -3.12 -m pip install pytest==8.3.5 ruff==0.14.13
py -3.12 -m pytest -o addopts= code/tests -q -p no:cacheprovider
py -3.12 -m ruff check code
py -3.12 code/run_demos.py
py -3.12 verify_deposit.py
py -3.12 tools/rebuild_release.py --check
```

On Linux or macOS replace `py -3.12` with the intended Python interpreter. The Rust parity test explicitly skips without a built binary; follow the README for native build commands. The proofs establish the lemmas under their assumptions. Their finite tests corroborate the stated domains; they do not prove implementation correspondence or a deployed effect.